Verification Guidance in Reused Agent Skills

Paul Revanth Madasu

CS 599 OSS Research Track

Professor Igor Steinmacher

Verification Guidance in Reused Agent Skills

Agent skills package instructions and optional supporting files for software agents. This study asks whether explicit, actionable verification guidance is associated with reuse across repository owners, and whether verification steps that refer to local supporting files are backed by the recorded folder inventory. It treats a distinct skill text as the primary unit and separates content reuse from the properties of a particular repository copy.

For this checkpoint, the public GitSkills sample was downloaded and queried locally in read-only SQLite. The database contains 29,786 file occurrences, 13,000 distinct contents, 11,841 repositories, and 47,829 sibling entries. All 13,000 representative texts independently reproduce their Git blob hashes. The audit also found 16 enriched nonrepresentative copies, 1,755 truncated representative folder listings, and sharply unequal history coverage across location classes. These observations determine the inclusion rules and missing-data policy below.

The strict study frame contains 11,192 content groups. A deterministic, stratified 240-group coding frame has been generated, but manual annotation and research-question comparisons have not been performed. The proposed contribution is a validated measure of verification guidance and an analysis of its relationship to reuse and supporting material, rather than another inventory of skill counts.

Evidence boundary. Sample queries and quality checks are completed. Full-release analysis, human labels, and hypothesis tests are planned. The study repository is linked in the work-plan section. No peer feedback was available; that requirement remains outstanding.

Dataset Overview and Available Resources

Dataset and release. GitSkills was created by Giuseppe Destefanis, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu to support empirical research on agent skills (Destefanis et al., 2026a, 2026b). The full archive is version 1.0.0, published August 10, 2026, DOI 10.5281/zenodo.21875637 (Destefanis et al., 2026a). This checkpoint queries its public sample, not the full release. Sources and sample were accessed October 23, 2026.

Exact analyzed snapshot. agent_skills_sample.db, 290,955,264 bytes, from the ZIP in sample repository revision fff3df93beeee51a5444450891751f143e38da3e (September 12, 2026). The ZIP was fetched through main and its Git blob hash was checked against that revision. Database and ZIP SHA-256 values are recorded in the reproducibility materials. The sample is not assigned a separate release number.

The full release reports 3,797,117 file occurrences and 1,877,981 distinct contents, compared with 29,786 occurrences and 13,000 contents observed in the sample. Published coverage is 282,200 repositories owned by 195,841 accounts; the sample contains 11,841 repositories owned by 10,786 accounts. The full release lists 7,264,865 sibling entries, including 3,497,752 with text. The sample contains 47,829 sibling entries, of which 22,278 contain text. There are 458,548 file histories in the published release and 3,010 in the sample. Both describe seven collection runs (Destefanis et al., 2026a; GitSkills Authors, 2026; Vaccargiu, n.d.).

Resources and format. The preprint (Destefanis et al., 2026b) explains the pipeline and structure; the sample README (GitSkills Authors, 2026) is the field dictionary and supplies working SQL examples. The Hugging Face card (Vaccargiu, n.d.) lists typed fields and a Python datasets loader. The full download is SQLite on Zenodo (44.4 GB) (Destefanis et al., 2026a), with table-sharded Parquet on Hugging Face (Vaccargiu, n.d.). This study uses SQLite for

transparent joins. No dedicated GitSkills notebook was verified; the verified examples are the README SQL and dataset-card loader. All URLs are in the references.

Collection and selection. Documentation describes read-only collection from GitHub code search, REST/GraphQL APIs, and raw content in July 2026. Searches were partitioned by file size to work around the 1,000-result limit. Content hashes group identical bytes; one preferred representative is enriched. Files lost before retrieval were excluded (GitSkills Authors, 2026; Vaccargiu, n.d.). The sample selects the 13,000 lowest content hashes and retains their occurrences and related records (GitSkills Authors, 2026). This approximates content-level sampling, not repository-level random sampling.

Dates and omissions. In the sample, discovery timestamps span July 9-20, 2026; first recorded commits reach back to April 14, 2025, while last commits extend through July 23, 2026. Collection dates describe retrieval, not creation. The data do not record agent invocation, execution success, user satisfaction, private repositories, or a complete edit history for every copy. Same bytes establish content identity, not authorship or copying direction.

License. The dataset card releases collected metadata and aggregation under CC BY 4.0; reproduced file text retains the origin repository license (Vaccargiu, n.d.). A missing repos.license is unknown permission, not a free-use designation. Reproduction materials here contain code, aggregates and identifiers; the raw dataset is downloaded separately.

Data Structure and Relationships

The actual SQLite schema has four substantive tables plus the internal sqlite_sequence table. PRAGMA inspection and executed joins support the following model; the full DDL is included with the reproducibility materials.

artifacts; One repository-path occurrence. Primary key (repo_full_name, path). file_sha links identical bytes. filename, location_class, discovered_at identify discovery context; dedup_primary identifies the selected text. content, frontmatter_valid, description, body_chars and composition/history flags support eligibility and measurement.

repos; One repository. Primary key full_name. Join artifacts.repo_full_name = repos.full_name. owner enables cross-owner counts; is_fork and metadata_fetched define known coverage; language, stars and license provide context.

artifact_siblings; One listed folder entry. Primary key (repo_full_name, artifact_path, entry_name). Join its first two fields to artifacts.(repo_full_name, path). entry_type distinguishes file from directory; content and entry_size describe recorded material.

mining_runs; One collector run. run_id identifies provenance; query, started_at, finished_at, discovered and note describe collection. No occurrence-level run foreign key is present.

Annotated Example From the Downloaded Sample

One representative is repository obot-platform/obot, path .claude/skills/draft-release-blog/SKILL.md, hash 00000f2a9d8359900f2075b536b435ca9473a5b9. It has name draft-release-blog, body_chars = 13,056, sibling_count = 1, has_scripts = 1, and has_references = 0. Joining on repository plus artifact path returns md_to_html.py, a file of 1,158 bytes. This is an observed metadata example, not evidence that the script runs or that the skill contains effective verification.

Repeated Content and Important Join Rules

There are 3,378 multi-occurrence groups and 2,391 groups spanning multiple owners among all 13,000 groups. A repository can contain several paths with identical text, and an

owner can control several repositories. Therefore COUNT(*) is an occurrence count, COUNT(DISTINCT repo_full_name) measures repository reach, and COUNT(DISTINCT owner) measures owner reach. None establishes independent adoption.

Filter text units by dedup_primary = 1, not content IS NOT NULL: 16 nonprimary rows also carry text. Join siblings to the chosen occurrence using the composite key, never file_sha alone. Representative folder contents must not be propagated to other copies. sibling_count agrees with file entries rather than all entries in complete inventories. The meanings of sibling content_fetched = 2 and missing skip reasons are not fully resolved; use actual text presence and explicit coverage flags.

Research Questions and Related Work

Verification Guidance and Cross Owner Reuse

Among eligible distinct skill contents, how does the prevalence of actionable verification guidance differ between skills held by one occurrence, multiple occurrences under one owner, and multiple owners? This matters because reusable instructions may spread more broadly than their safeguards. A noncausal association can identify whether widely distributed skill texts deserve targeted review for clear checks, without equating popularity with correctness.

The unit is one file_sha content group. Required fields are representative content and body_chars; all eligible occurrences provide file_sha and repo_full_name; repos.owner supplies distinct-owner reach. Human labels measure verification specificity. The main comparison estimates the difference in actionable-guidance prevalence between the cross-owner and single-occurrence strata, with a same-owner stratum as a separate comparison.

Supporting Files for Verification Steps

For eligible representative skills whose verification steps explicitly reference local supporting files, how often are those files present in the recorded complete folder inventory, and does support differ by verification specificity? This connects instruction quality to the material available to carry it out. The unit remains a content group represented by one repository-path folder; it is not every copy of that content.

Required fields are representative content, composition_fetched, composition_truncated, and sibling entry_name and entry_type. Manual coding isolates local file references associated with a check. The analysis compares the fraction of fully resolved local references for V2 versus V3 guidance and reports unresolved cases separately. No claim about execution success is possible from file presence alone.

What Existing Work Establishes

GitSkills (Destefanis et al., 2026b) establishes a corpus and a distinction between file occurrences and distinct content. Its data enable static reuse analysis but do not measure whether an agent follows a check. Destefanis (2026) argues for evaluation-driven skill authoring and behavioral evaluation. That motivates a verification construct but does not establish its prevalence or reuse association.

Gao et al. (2026) empirically study authoring, reuse and maintenance in registry and personal-use skills. They report substantial near-verbatim reuse and limited changes to behavioral contracts. Their broader content and modification analysis motivates examining explicit verification guidance with a reproducible, narrower rubric.

Seddik (2026), submitted October 8, 2026, reconstructs dated copy networks, studies propagation and risk, and evaluates audit ranking. That work makes a generic copying-

concentration study insufficiently distinctive here. This project instead examines verification instructions and their local supporting references; it does not reconstruct diffusion or infer source repositories.

Kalliamvakou et al. (2014) show that GitHub repositories have heterogeneous purposes and that mined traces can misrepresent activity. Accordingly, repository counts are not treated as production use, and catalogs/templates will be recorded as contextual categories. The open format specification (Agent Skills, n.d.) requires metadata and allows arbitrary body instructions and optional supporting directories; valid front matter is therefore an eligibility proxy, not a quality judgment.

Expected addition. A small, auditable codebook; a human-labeled stratified sample; effect estimates for verification guidance and reuse; and a conservative check of local supporting-file presence. Novelty is scoped to this combination within the reviewed sources; no claim of exhaustive literature coverage or first-ever discovery is made.

Proposed Method

Design. A mixed quantitative and qualitative, cross-sectional study of the pinned GitSkills sample. The sample alone is the committed semester scope. A full-release replication is optional and depends on storage and time; it will not replace the pinned sample silently. The audit and feature extraction below are completed; semantic coding and inferential comparisons are proposed.

Population and Selection

The primary frame keeps dedup_primary = 1, exact case-sensitive filename SKILL.md, frontmatter_valid = 1, and nonempty content, joined to a repository with metadata_fetched = 1 and is_fork = 0. There are 11,197 exact-name, parser-valid representatives; five have unknown

fork status and are excluded, leaving 11,192 groups. The parser flag does not certify every constraint of the current specification. A sensitivity analysis will relax filename/header restrictions and show exclusions explicitly.

Reuse is computed over occurrences with the exact basename and known nonfork repository status, grouped by file_sha. All three strata refer to this restricted occurrence set: 8,265 single-occurrence, 693 same-owner multiple-occurrence, and 2,234 cross-owner groups. The representative need not be the first or original author. Unknown repository status is not recoded as nonfork.

Select 80 groups per stratum (240 total), ordered within stratum by SHA-256 of cs599-label-v1: followed by file_sha. This selection was executed and the blank coding form is included. Hide strata, reach counts and stars during coding. The design overrepresents reused groups to enable comparison; frame-level prevalence uses weights N_h/80. The unequal stratum counts do not change within-stratum estimates.

Operational Definitions

Verification specificity will be coded at four levels. V0 means there is no directed output check. V1 means the skill makes a generic request to test or review. V2 means it gives a concrete check action or an observable acceptance criterion. V3 means it provides both and explicitly connects them. V2 and V3 form the primary actionable-guidance category. Uncertain cases will remain unresolved rather than being forced into V0.

Reuse will be measured using occurrences, distinct repositories and distinct owners separately. Cross-owner reuse means that eligible occurrences span more than one owner. Manual coding will also note whether the skill concerns a software task and whether its context is a catalog or template. Recorded presence does not establish invocation.

Supporting material will be checked only when a verification step explicitly refers to a local file. The relative path will be normalized and matched to a file entry in that representative folder. The result will distinguish recorded presence, absence in a complete inventory, and unknown or ambiguous status.

Question to Analysis Mapping

RQ1: file_sha unit + human guidance label + owner/occurrence reach. Report stratum proportions, their cross-owner versus single-occurrence risk difference, and an odds ratio. Generate 95% intervals by 10,000 within-stratum bootstrap resamples (fixed seed 599); use weighted summaries only when combining strata. Treat the third stratum as a secondary comparison.

RQ2: representative folder unit + human-linked local paths + inventory coverage. Among complete folders with eligible references, compare per-skill resolved-reference fraction for V2 versus V3 using a bootstrap difference in means, resampling skills rather than individual references. Report denominators, uncertainty and examples; use descriptive results when either group has fewer than 20 skills. No manual labels or effect estimates exist yet.

Measurement Validation and Interpretation

Human Coding and Checks

Pilot 30 of the 240 selected groups, refine ambiguous rules, freeze codebook v1, and recode those 30 before final analysis. Read the whole skill, distinguishing directed checks from background prose and illustrative user tasks. Record evidence line numbers and reasons for each label. Invented codebook examples: "review the result" is V1; "run pytest and require all tests to pass" is V3. These examples are not reported dataset findings.

Blindly recode 60 items after at least seven days: the 30 pilot items plus 30 additional deterministic selections. Report raw agreement, weighted Cohen kappa for V0-V3, and uncertainty. This is intra-rater stability, not independent inter-rater reliability. If weighted kappa is below 0.60, clarify rules and recode affected cases; if stability remains weak, collapse to actionable/nonactionable and limit interpretation. An independent second reader is desirable but is not available or claimed.

For RQ2, independently recheck every coded absent-file case against the composite-key inventory join and the literal reference context. Exclude generated outputs, placeholders, absolute paths, URLs, and paths clearly belonging to the user's external project. Mark ambiguous bases and symlink references unknown; never classify a missing text body as a missing file. An existing entry establishes recorded presence only, not that it is executable, safe, current, or usable.

Automation policy. No LLM classifier is used for reported findings. SQL and deterministic hashing handle structure and selection. If an LLM is added later, reserve an untouched human-labeled holdout and publish the model/version, prompt, confusion matrix, per-class precision/recall and abstentions. Agreement between models will not substitute for human validation. The prepared coding frame contains no completed semantic labels.

Analysis Safeguards

As an exploratory adjustment for RQ1, fit cross-owner versus single-occurrence logistic regression using actionable guidance, log(1 + body_chars), and a software-task indicator only if data support at least ten observations of the less frequent outcome per coefficient and no separation. Otherwise keep unadjusted contrasts. Do not search many model specifications for significance. Inspect a sensitivity analysis excluding catalogs/templates and another retaining

one representative per owner; explain that representative ownership does not capture every dependency across shared owners.

For unresolved labels, report their count and repeat prevalence contrasts with unresolved cases assigned to each extreme. For incomplete folder inventories, give both complete-case estimates and worst-case presence bounds over eligible references. Do not impute absent support from a truncated listing. These are planned sensitivity checks, not results.

Threats Scope and Ethics

The snapshot cannot establish that verification guidance causes reuse or that it improves outputs. Shared templates, task purpose, text length and repository selection may confound associations. Exact hashes miss edited and near-duplicate skills. Hash-based content sampling is not a probability sample of repositories, owners or all GitHub activity. Header validity excludes some genuine skills and may include unrelated instructions. File history is path-specific and heavily selected, so maintenance is outside the primary RQs.

Analyze instructions as text; do not execute downloaded commands or attempt to reverse anonymized author codes. Report aggregates and short, license-aware examples. A missing check or support file is not evidence of malice. This report was prepared with AI assistance for source retrieval, analysis execution and writing; semantic research labels remain to be completed and reviewed by the student.

Initial Data Quality Checks

All figures in this section are actual results from the pinned sample, produced by checks.sql and run_checks.py. They describe this sample only. The companion results file includes denominators, runtime versions and the complete seven-row collection log.

SQLite integrity checking returned "ok." Independent Git blob hash checks found zero mismatches among the 13,000 representative contents, and no artifact had a missing content hash. These checks support byte identity for the sampled texts, although they do not validate meaning or execution.

Relationship checks found no artifact-to-repository or sibling-to-artifact orphans. Each of the 13,000 content groups had exactly one primary representative. The schema declares primary keys but no foreign-key constraints, so the analysis uses explicit joins and checks their results.

Among the 16,786 nonprimary records, 16 also contained text and composition information. Thirteen sibling records joined to nonprimary occurrences. The reason for this enrichment remains unresolved. The study therefore explicitly selects dedup_primary = 1 for content units and joins supporting files to those selected occurrences.

There were 378 occurrences among the 29,786 records whose filename differed from the exact spelling SKILL.md. Among the 13,000 representatives, 1,709 had frontmatter_valid = 0. The main analysis uses exact filenames and parser-valid primary contents, with broader inclusion planned as a sensitivity check.

Ten of the 13,000 representatives lacked fetched composition information, and 1,755 inventories were truncated, representing 13.50% of all representatives. In the strict study frame, 10,079 of the 11,192 groups had fetched, untruncated composition. Missing entries in incomplete inventories will not be interpreted as proof that a file is absent.

Comparing sibling_count with all recorded entries produced 2,915 mismatches for representatives with fetched, untruncated inventories. Restricting the comparison to file entries reduced the mismatch count to zero. Directory rows explain the apparent discrepancy, so comparisons with sibling_count will count files only.

Text was present in 22,278 of the 47,829 sibling entries. The table contained 39,046 file entries and 8,783 directory entries, and all skipped_reason values were missing. Seventy-one file records had content_fetched = 1 but no content. Text availability will therefore be determined from actual content presence, while the reasons for missing text remain unverified.

Among the 11,841 repositories, 728 lacked a language value and 6,331 lacked license information. Fork flags and star counts were unknown for 37 repositories, while no owner values were missing. Unknown categories will be retained explicitly. Five otherwise eligible primary units with unknown repository fork status were excluded from the main frame, and missing licenses will not be interpreted as permission to reuse file contents.

All present discovery, first-commit and last-commit timestamps could be parsed, and no first-commit timestamp was later than its corresponding last-commit timestamp. Two recorded histories preceded October 2025. Four collection runs were closed retroactively with zero counts. Timestamp comparisons use SQLite julianday rather than lexical ordering, and collection-run counts will not be summed as a corpus census.

Uneven history coverage. Histories exist for 1,755/1,762 canonical representatives (99.60%), 813/7,416 skills-dir representatives (10.96%), and 442/3,822 other representatives (11.56%). Treating the 3,010 available histories as a random content sample would bias maintenance comparisons. The main study therefore does not require commit-history fields.

Risks still untested: near duplicates, symlink prevalence, correctness of front-matter parsing, true reference resolvability, label agreement, and success of any verification action. No prevalence or causal claims about these are reported.

Work Plan and Responsibilities

The current plan assigns responsibility for provenance, SQL, manual coding, analysis, interpretation, writing and the oral explanation to Paul Revanth Madasu. If a second member is added, responsibilities will be divided explicitly while both members review the full design. No second coder or independent two-person validation is claimed.

Oct. 23-26. Finalize checkpoint package; review SQL and report; verify the public repository link; request feedback from another group. Student leads all tasks. Repository access and peer availability. Submit the honest checkpoint version under the available submission process.

Oct. 27-Nov. 2. Pilot 30 labels, freeze codebook, code remaining 210, validate reference matching, prepare descriptive summaries. About 20-25 hours for coding at 5-6 minutes per item. If over budget, freeze a reduced balanced sample of 40 per stratum and report reduced precision.

Nov. 3 progress report. Show audit, pinned sample, codebook, annotation progress, initial stratum estimates and explicit unknowns. Demonstrate a reproducible query. If coding is incomplete, present completed-label denominators and a completion date; do not label planned work as done.

Nov. 4-17. Blind recode after seven days; resolve disagreements; run planned contrasts and sensitivity checks. Optional full-release replication only after sample analysis. Second reader and 44.4 GB download are optional. Keep sample-based claims if storage or access blocks replication.

Nov. 18-Dec. 2. Complete interpretation, charts, limitations and reproducibility audit; revise after available peer feedback; rehearse. If reference resolution proves ambiguous, narrow RQ2 to hand-verified complete folders and report unknown cases.

Dec. 3 presentation. Present questions, observed data-quality findings, final sample results, uncertainty and limitations; explain units and joins. Student must be able to rerun SQL and explain label decisions.

Dec. 4-9 / Dec. 9 paper. Incorporate presentation feedback, verify claims against outputs, finalize paper and archive code/manifest/results. Confirm the public repository link and preserve the final analysis version.

Peer Feedback and Response to Review

Feedback from another group has not yet been received for this report. Once it is available, the response will identify the group and review date, summarize each comment, describe the resulting revision and its location, and explain any suggestion not adopted. The scope decisions and safeguards described here are preparation decisions rather than revisions attributed to peers.

Study repository. https://github.com/paulrevanthpersonal-lab/cs599-gitskills-research. The public repository contains the read-only analysis runner, executed SQL, observed results, snapshot manifest, proposed codebook, blank coding frame and checkpoint report. The repository can be inspected without signing in. The raw dataset is downloaded separately from the authors' public source (GitSkills Authors, 2026). A local reproducibility ZIP also accompanies the report.

The in-class check-in remains a separate course requirement. The planned demonstration will show the sample row-count query, the representative invariant and the sibling-file count check in the repository. The explanation will distinguish content groups from occurrences and describe how the proposed labels will be validated. The check-in is not represented as completed.

Reproducibility and Execution Record

Completed environment. Python 3.12.14 with SQLite 3.53.1, using only the Python standard library for data analysis. The SQLite connection uses mode=ro. run_checks.py executes the named SQL blocks, writes JSON results and the actual schema, independently recomputes representative Git blob hashes, extracts the strict study frame, and creates a blank 240-row coding form. It never executes dataset contents.

Download and Identity

Sample source: authors' sample repository. Download agent_skills_sample.zip at the pinned revision in the pinned sample repository (GitSkills Authors, 2026), then unzip it. The archive contains one database. The original download was verified against the revision's Git tree, which records ZIP blob SHA-1 b2c5b19033285d2361a20df94d36e3eb922b6a1a.

The downloaded ZIP contains 86,721,947 bytes and the database contains 290,955,264 bytes. Exact SHA-256 checksums for both files are recorded in manifest.json and the repository README. The database checksum begins 683888c923a51952 and ends 7592ae27c367658; the full checksum must be used for verification.

Portable Rerun Commands

To reproduce the analysis, download and unzip the pinned sample as explained in the repository README. From the repository folder, run python3 run_checks.py agent_skills_sample.db --out rerun-results, supplying the correct database path. The runner opens the database read-only and writes new local outputs. A simple initial SQL query is SELECT COUNT(*) FROM artifacts;, which returned 29,786. The files checks.sql and study.sql contain the full executed quality checks and feature-extraction SQL.

The Python runner needs no SQLite command-line installation. It accepts any database path, exports results locally, and records the database hash and SQLite runtime version. A new release or modified database requires a new manifest and rerun; results should not be compared under an unchanged snapshot label.

Companion Archive Contents

The repository and companion archive preserve checks.sql and study.sql, which contain the executed quality queries and content-group extraction. The standard-library script run_checks.py reruns the audit and independent content-hash checks. The manifest records snapshot identity, checksums and source URLs. The observed query outputs and actual database schema appear in results/observed_results.json and results/schema.sql. The study summary records the 11,192 eligible groups and 10,079 complete inventories. The proposed codebook and 240-row coding form are included separately, with semantic labels left blank.

Hash audit detail. For each primary content string, encode UTF-8, prepend the Git header bytes blob + space + byte-length + NUL, and calculate SHA-1. Compare to file_sha. All 13,000 matches are independent of the dataset's content_sha_ok flags. A hash match verifies retained bytes; it does not validate source licensing, semantics, or folder content.

The sample README describes full archival as future work, while the official call, Zenodo record and Hugging Face card provide the full release links (Destefanis et al., 2026a; MSR 2027 Organizers, n.d.; Vaccargiu, n.d.). Accordingly, the report uses those sources for current availability. The README's approximate sizes are distinguished from the observed sample byte count and Zenodo's displayed 44.4 GB. Full-release counts remain published figures because the complete database was not queried.

References

Agent Skills. (n.d.). *Specification.* Retrieved October 23, 2026, from https://agentskills.io/specification

Destefanis, G. (2026). *Authoring agent skills: A software-engineering approach* [Preprint]. arXiv. https://arxiv.org/abs/2607.25032v1

Destefanis, G., Graziotin, D., Vaccargiu, M., & Ortu, M. (2026a). *GitSkills: A dataset of agent skills on GitHub* (Version 1.0.0) [Data set]. Zenodo. https://doi.org/10.5281/zenodo.21875637

Destefanis, G., Graziotin, D., Vaccargiu, M., & Ortu, M. (2026b). *GitSkills: A dataset of agent skills on GitHub* [Preprint]. arXiv. https://arxiv.org/html/2608.10906v3

Gao, H., Lulla, J. L., Lin, H. Y., Baltes, S., Treude, C., & Zahedi, M. (2026). *From registry to repository: How AI agent skills are written, adapted, and maintained* [Preprint]. arXiv. https://arxiv.org/abs/2607.00911v2

GitSkills Authors. (2026). *GitSkills sample* [Data set documentation; revision fff3df93beeee51a5444450891751f143e38da3e]. GitHub. https://github.com/giuseppedestefanis/gitskills-sample/tree/fff3df93beeee51a5444450891751f143e38da3e

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., & Damian, D. (2014). The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories* (pp. 92-101). Association for Computing Machinery. https://doi.org/10.1145/2597073.2597074

MSR 2027 Organizers. (n.d.). *Mining challenge.* Retrieved October 23, 2026, from https://conf.researchr.org/track/msr-2027/msr-2027-mining-challenge

Seddik, F. (2026). *Skill constellations: Tracing the supply chain of agent skills on GitHub* [Preprint]. arXiv. https://arxiv.org/abs/2610.11169v1

Vaccargiu, M. (n.d.). *GitSkills* [Data set documentation]. Hugging Face. Retrieved October 23, 2026, from https://huggingface.co/datasets/mvaccargiu/gitskills/blob/main/README.md